

Hold My Wheel

Development Plan

Version 0.1

09/12/2025

Authors

Rayan Naboulsi
Miquel Plana-Piñol
Christophe Picard
Keija Louis-Marie
Safae Salhi
Caroline Nguyen
Adam Salmi

1. Introduction

The Software Development Plan aims to provide the approach for the development, verification and validation of *Hold My Wheel*. It highlights our project management and development approach, software engineering standards and techniques, our development and testing environment and our documentation plan.

2. Applicable and reference documents

All the documents related to the project can be found on the [Kleya Group Website](#).

Additionally, other documents are available on the Kleya project  geicar - Kleya .
Mainly, these documents are relevant :

- The [agile/scrum manifesto](#)
- The [scrum guide](#)
- The [gitflow workflow application guide](#)
- The [project plan](#)
- The [project architecture notes](#)

3. Terms, definitions and abbreviated terms

- ADAS - Advanced driver-assistance system
- ESP - Electronic Stability Program
- FCTA - Front Cross Traffic Alert
- RCTA - Rear Cross Traffic Alert
- LDW - Lane Departure Warning
- LKA - Lane Keeping Assist
- ACC - Adaptive Cruise Control
- AES - Automatic Emergency Steering
- HMI - Human Machine Interface
- IMU - Inertial Measurement Unit

4. Software project management approach

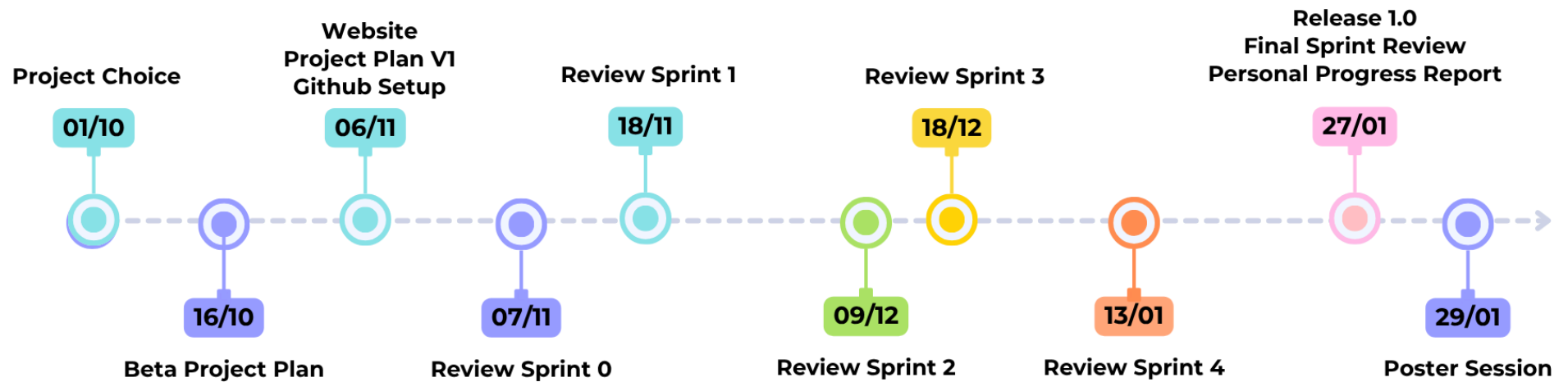
4.1. Management objectives and priorities

The software development is conducted using an Agile–Scrum inspired workflow adapted to an academic project. The objective is to implement an ADAS (Advanced Driver Assistance System) on the GEI car in order to improve user safety. The project is structured around short iterative sprints, each delivering incremental and testable features. Prioritization follows a safety-first approach: the features that directly affect user protection and vehicle stability are delivered first.

ID	Feature	Priority (0=High)
[1]	Collision Avoidance (FCTA/RCTA)	0
[2]	Airbag	1
[3]	Trajectory Control Assistance (ESP)	3
[4]	Web HMI with malfunctions and notifications	2
[5]	Lane Centering Assist (LDW/LKA)	5
[6]	Speed Control (ACC)	4
[7]	Emergency Automatic Obstacle Avoidance (AES)	5

4.2. Master schedule

Our project uses an Agile based schedule, with 6 sprints from October through January.



4.3. Assumptions, dependencies and constraints

- The software development phase ends on 27th January 2026, as set by the global project schedule.
- Each sprint lasts 10 days to 2.5 weeks, depending on workload and academic constraints.
- All team members have access to the hardware platform (GEI car, sensors, Raspberry Pi, Git, Jira).
- The project relies on stable versions of ROS2, GitHub, and the existing car control framework.

The project depends on external events :

- Delivery of hardware modules (IMU, ultrasonic sensors, radar, boussole, steering servo).
- Faculty validation at each milestone (Sprint Reviews, Poster Session).
- GitHub availability for version control and continuous integration.

It also has technical issues that have an effect on the plan :

- Sensor noise (IMU drift, ultrasonic false readings, compass calibration).
- Network instabilities between Raspberry Pi and PC over ROS2.
- Power limitations or battery constraints during long test sessions.

4.4. Work breakdown structure

We broke down our feature into user stories with an associated priority. This approach allows us to focus on the delivered value to the client and their priority. The user stories are organized in a table with a unique identifier as it follows. In the table below, a priority of 1 is highest.

Name	User Story (Description)	Priority
[1] Collision Avoidance (FCTA / RCTA)	As a driver, I want the vehicle to detect nearby obstacles and gradually reduce speed until it stops if necessary to avoid collisions with surrounding objects.	1
[2] Airbag	As a driver, I want the vehicle to deploy airbags in the event of a collision so that I can be protected from head and neck injuries.	1

[3] Trajectory Control Assistance (ESP)	As a driver, I want the vehicle to detect when it loses its intended trajectory and automatically adjust its steering so that my car stays centered.	2
[4] Lane centering assist (LDW/LKA)	As a driver, I want the vehicle to automatically stay within the lane so that I can safely stay inside the road limits and avoid possible accidents.	2
[5] Human Machine Interface	As a driver, I want the car's Human-Machine Interface to display real-time information about speed, and other key vehicle parameters, so that I can monitor the car's performance at any moment and be alerted immediately in case of emergencies such as airbag deployment or collisions.	1
[6] Detection of malfunctions and notification	As a driver, I want to be notified when one of my ADAS systems is malfunctioning so that I can remain aware of the status of my vehicle and can locate and remediate malfunctions.	3
[7] Speed Control (ACC)	As a driver, I want to be able to maintain a constant speed so that I do not have to regulate it myself. As a driver, I want my ACC to adapt the speed of the car according to the traffic in front of me so that I can maintain a safe distance without constantly breaking and accelerating.	4
[8] Emergency automatic obstacle avoidance (AES)	As a driver, I want my car to automatically avoid obstacles so that I can avoid obstacles even when I do not react in time.	5

4.5. Monitoring and controlling mechanisms

To ensure the project progresses as expected, the team uses:

Task tracking

- Each sprint backlog is defined in Jira.
- Features are tracked from To Do -> In Progress -> Testing -> Done.

Progress monitoring

- Continuous integration through GitHub.
- Regular testing of ROS2 nodes on simulated or real hardware.
- Weekly meeting to coordinate feature implementation

We also have mechanisms to facilitate the follow up for our client such as:

Progress monitoring

- Weekly progress meetings with the team and supervisor.
- Sprint reviews validating completed features.

Milestone tracking

- Delivery of functional increments at each sprint review.
- Alignment with major project milestones (e.g., Review Sprint 0, etc, Poster Session).

4.6. Staffing plan

The team does not use fixed roles. All members have similar skills, so the workload is shared evenly.

At the start of each sprint, the team proposes the upcoming tasks and holds a vote so each member can choose the features they want to work on. We update what is done on the group chat. This ensures motivation and a fair distribution.

Tasks are usually handled by 1–2 members. This flexible organisation allows everyone to contribute to software development, sensor integration, testing, and documentation.

Overall, the staffing approach is collaborative, adaptive, and consistent with the Agile workflow used throughout the project.

5. Software development approach

5.1. Strategy to the software development

Our project is based on many different systems that are independent from one another. Therefore, we have decided to assign features (that mostly represent individual systems and consist of one or more user stories) to teams. Their backlogs will then be the different functionalities of each feature. This is not very well reflected in the first sprint planning, as its main goal is that everyone in the project is acclimated with the GEI-Car project.

Agile methodology is important in the scope of our project as we are dealing with many shifting objectives, whether it is in the way that the client's needs are unclear or in the way that the capabilities of the car are also vague. Using Agile and this project plan, at the end of each sprint, we will be able to accurately measure how much of every feature has been completed, which features to focus on, etc.

5.2. Software project development life cycle

5.2.1. Software development life cycle identification

The project will follow an Agile life cycle, with 5 sprints from October 2025 to January 2026. The life cycle will be organized with these short and iterative sprints. Each sprint includes planning, implementation, verification, validation, and review activities.

The project uses an incremental life cycle. Functionalities are developed progressively in manageable increments and each increment is validated using a validation plan.

Software versioning is handled in the following way, using GitFlow :

- main : contains review-ready releases
- develop : integration branch for completed features
- feature-* : branches used for individual functionalities
- hotfix-* : branches used for urgent corrections
- patch-* : branches used for the additions or modifications of the underlying architecture

At the end of every sprint review, version tags are applied to completed releases.

Requirements are collected and prioritized during the first sprint (Sprint 0). They are then compiled into a Project plan and refined during each sprint. Architectural decisions evolve throughout the design and are reviewed by all team members through regular team meetings.

Design for each feature is done within the sprint according to the product backlog. Every feature is implemented within its own branch, and peer review is done through the regular team meetings as well as the develop pull requests.

Coding standards guide implementation.

Verification is performed continuously using :

- SonarQube Cloud connected to the github, performing automated code analysis
- Regular code reviews
- Regular team demonstrations
- Regular client demonstrations

Only the features that have been verified are merged into develop.

Validation occurs at the end of each sprint. Completed features are demonstrated and reviewed during the sprint review. We thus ensure that the feature satisfies the client's needs. Additionally, the product owner approves each feature during regular team meetings.

At the end of each sprint, when features are validated, the release is merged into main with a new tag. Functionalities are delivered to the stakeholders following the release version. Acceptance is granted by the clients.

5.2.2. Relationship with the system development cycle

System development and software development are done in parallel. Every software architecture decision and requirement is shaped by the system. Each software increments specific system functions being developed in the same timeframe. Validation tests are done on the physical system directly with verified code.

In the following table, we are listing the different features of the project and the hardware material associated. Most of the hardware in this system is part of the GEICar platform, except for the computer connected for the HMI and the Electronic compass.

Name	Hardware Used
[1] Collision Avoidance (FCTA / RCTA)	- Front and back ultrasonic sensor
[2] Airbag	- IMU (X-Nucleo-IKS01A2)
[3] Trajectory Control Assistance (ESP)	- IMU (X-Nucleo-IKS01A2) - Electronic Compass
[4] Lane centering assist (LDW/LKA)	- Camera connected on the jetson
[5] Human Machine Interface	- A computer connected to the switch

[7] Speed Control (ACC)	- Ultrasonic sensors
-------------------------	----------------------

5.2.3. Reviews and milestones identification and associated documentation

The project uses 5 sprints planned about every 2 weeks. Each sprint monitors progress, ensures client feedback and satisfaction and validates product releases. Each review focuses on the advancements made within the sprints.

Points addressed	Deliverables
Next sprint goals and validation plan	20-min presentation with clients In-person tests with clients Test evidence Release version
Review backlog changes and adjustments	
Assess architectural changes and efficiency	
Confirm that the delivered software meets requirements and validation plan	
Evaluate team performance and efficiency	Retrospective notes
Design and architectural decision-making	Design documents Architectural notes

There are two types of technical reviews :

- Client reviews, which are very formal and non-specialised
- Team peer code reviews, which are semi-formal to non-formal and very technical

At each review, is present :

- Ms. Moore, representing INSA as an english communication instructor.
- Mr. Di Mercurio, representing the technical support of the project at INSA and one of the clients
- Mr. Auriol, representing INSA and one of the clients

INSA acts as a funding and resource for the project.

5.3. Software engineering standards and techniques

The project contains different programming language depending on the module :

Language	Use	Standard/Reference
C/C++	Low level driver nodes (IMU, ecompass...)	Clean code principles (Robert C.Martin) C++17 standards
	Nucleo programming	MISRA-C
Python	ADAS Nodes	PEP 8 style guide for formatting and naming
HTML/JS	HMI	W3C HTML5 recommendations

Across all languages, common rules are applied :

- No unused code
- Limited function complexity
- Clear error handling and logging

The project uses the following methodologies :

- Agile methodology
 - Used as the backbone for the management and development framework.
 - User stories, features and validation plans follow Agile methodology.
- Gitflow workflow
 - Used for the version control and branching strategy.
 - For description : see [the atlassian gitflow tutorial](#).
- Static code analysis
 - Using SonarQube Cloud, connected to our Github.

The design used for our project is one based on modularity. The focus is on making each feature and functionality independent to facilitate the development and testing. Our architecture in ROS2 is based on a central node acting like the supervisor of the ADAS functions. This central node gives priority to each function and sends the command to the car accordingly. Each ADAS function is connected to the rest of the system via the supervisor node and can be developed individually. In addition to this base architecture, we have an HMI connected to the supervisor node to get the information to display.

Software delivery is handled through git, with the appropriate version tags using the following format v<sprint>.<revision>.<hotfix>

For example, v1.0.1.

Additionally, we will use git for version control, using gitflow. Each feature is implemented on a separated branch, and regrouped with the main branch when they are properly implemented, tested and validated.

5.4. Software development and software testing environment

The project uses a controlled development environment, using Agile/Scrum and gitflow.

- Software development environment

The software development environment is a ROS2 workspace, building upon and modifying an existing architecture (gei-car). This environment is thus built upon using C++ and Python. Most features are implemented using Python, with drivers and interfaces developed using C++.

Python was selected for its ease-of-use, modular design and compatibility with existing libraries. C++ was selected in drivers and interfaces for its reliability and speed.

- Software testing environment

Each branch has its own separated testing environment on the hardware. On the target hardware, each branch is in its own ROS workspace. In their own workspace the following tests are concluded :

- Unit tests, using pytest
- Static analysis, using SonarQube
- Functional tests, following the validation plan, to validate the operational requirements

All code is written on standard machines, for their use of IDEs, support for git and support for the testing infrastructure. It is then compiled once on the machine running ROS2 Humble on Linux, where the car can be simulated for a first verification.

The IDE used is VSCode, selected for its compatibility and extensive extension support, including GitGraph, for visualising our Gitflow architecture. We also use the built-in ROS build and dependency tool, colcon. Additionally, we use pytest for unit testing our python nodes.

The code is then properly verified and validated when it is implemented on the target hardware directly. Using colcon, the code is then compiled directly on the Raspberry PI 4.

All development and testing tools were selected for the following reasons:

- They support Agile/Scrum iterative development.
- They integrate seamlessly with GitFlow and automated testing
- They ensure reproducibility, traceability, and maintainability throughout the software life cycle.

5.5. Software documentation plan

5.5.1. Software documentation identification

The project documentation follows these principles:

- Standardized formatting and structure (consistent templates, headings, versioning, authorship)
- Naming conventions ensuring consistency across all documents
- Traceability between requirements, design, implementation, test procedures, and validation evidence
- Review and approval workflow aligned with the project's Agile/Scrum governance and quality assurance process

Applicable documentation standards include:

- Internal document templates
- Versioning rules consistency with GitFlow
- Feature/Functionality/Requirement naming conventions

5.5.2. Deliverable items

Name	Description	When delivered	Format
Project architecture notes	Highlights the ROS2 architecture of the project	End of Sprint 5	PDF
Project plan	Regroups feature and functionality description, operational requirements, validation plan Follows Agile/Scrum methodology	End of Sprint 0	PDF
Development plan	Defines the software features, functionalities, operational requirements, and the overall validation approach. Describes how development is organized and executed following an Agile/Scrum methodology.	End of Sprint 5	PDF
Product assurance plan	Describes the quality, compliance, and assurance measures applied throughout the project to ensure the software meets required standards and delivers a reliable, validated product.	End of Sprint 5	PDF
Github Readme	Describes product use and functionality, in a technical format.	End of Sprint 5	Markdown

Hold My Wheel

Product Assurance Plan

Version 0.1

09/12/2025

Authors

Rayan Naboulsi
Miquel Plana-Piñol
Christophe Picard
Keija Louis-Marie
Safae Salhi
Caroline Nguyen
Adam Salmi

1. Introduction

The Software Product Assurance Plan (SPAP) explains how we make sure that the software developed for the project *Hold My Wheel* autonomous driving system is safe, reliable, high-quality, and developed in a controlled and traceable way.

The goals of this SPAP are to:

- ensure that quality assurance is included in every phase of the software development,
- define rules and methods used to check the software quality,
- identify the standards, tools, and procedures followed by the team,
- ensure that all software delivered is consistent with requirements, correctly tested, and documented,
- monitor risks, errors, and non-conformities during the project.

2. Applicable and reference documents

The following documents are used as references for this SPAP:

- The [Project Plan](#)
- The [Software Development Plan](#)
- ECSS-Q-ST-80C — Software product assurance standard
- ECSS-E-ST-40C — Software engineering standard
- ECSS-M-ST-10C — Project management and planning
- Hold My Wheel — [System Architecture Document](#)
- [Sensor datasheets](#) (IMU, ultrasonic sensors, camera)
- ROS2 documentation
- [GitHub repository](#)

These documents define the rules, constraints, and references used to check and assure the quality of the software.

3. Terms, definitions and abbreviated terms

To keep consistency across project documents, all terms and abbreviations used in this SPAP come from the Software Development Plan (SDP).

Examples of common terms used in this plan:

- **ADAS** → Advanced Driver Assistance System
- **ACC** → Adaptive Cruise Control
- **ESP** → Electronic Stability Program
- **IMU** → Inertial Measurement Unit
- **ROS2** → Robot Operating System 2
- **FMECA** → Failure Modes, Effects and Criticality Analysis
- **FTA** → Fault Tree Analysis
- **CI/CD** → Continuous Integration / Continuous Deployment
- **DoD** → Definition of Done

4. System overview

The *Hold My Wheel* project improves the GEICar by adding several ADAS (Advanced Driver Assistance System) features. These features include collision avoidance, airbags, trajectory control, lane assist, and automatic avoidance. They help the car detect obstacles, stay in its lane, control its speed, and respond safely in different situations.

The system is made of two main parts: hardware and software.

4.1. Hardware architecture

The system relies on several interconnected hardware components, each with a specific role:

Components	Description
Raspberry Pi	• Serves as the main computer in the car. • Runs the ROS2 programs. • Receives sensor data, makes ADAS decisions, and sends commands. • Communicates with the STM32 microcontroller.
Jetson Nano	• Runs other ROS2 Nodes • Handles GPU-heavy applications such as : ◦ Camera applications ◦ AI models
STM32 NucléoF103RB	• Handles the fast and critical tasks. Directly controls the motors, steering, and related components. • Executes the commands sent by the Raspberry Pi.
Sensors	• Ultrasonic sensors: detect nearby obstacles. • IMU: measures the car's movements and orientation. • Camera: helps with lane keeping. • Accelerometer (impact sensor): used for safety and airbag triggering. Electronic compass: used for detecting skidding and lane loss. • Depending on the sensor, the data is sent either to the STM32 or to the Raspberry Pi.

4.2. Software architecture

The software is organized into several ROS2 packages that work together to manage the car. Each package has a specific role:

- **Perception:** receives and processes all sensor data.
 - **Control:** uses this information to adjust speed, steering, and trajectory.
 - **Safety:** monitors risks, detects collisions, and triggers emergency actions.
- Interface:** displays all useful information to the user on the dashboard.

These packages contain nodes (executables) that communicate the needed data through ROS2 topics. Sensor data flows into the perception layer, decisions are made by the control and safety layers (nodes), and commands are sent to the STM32.

Finally, the dashboard shows the current state of the vehicle in real time (HMI).

4.3. Interaction between hardware and software

The sensors send their data to the Raspberry Pi and/or the STM32.

The STM32 handles everything that must react very quickly: motors, steering, and airbags.

The Raspberry Pi analyzes the data, makes decisions based on the program's logic in the nodes, and sends commands to the STM32.

Both units exchange information continuously through a communication link (UART, CAN, or USB).

The Streamlit Dashboard displays:

- the speed,
- the status of the sensors,
- alerts,
- and the current ADAS actions.

In summary:

Sensors → Analysis → Decision → Action → Display

5. Software product assurance programme implementation

5.1. Organization

Our organizational structure follows :

- **Scrum Master** – ensures that the team follows agile practices, facilitates meetings, removes obstacles, and supports quality and process compliance.
- **Product Owner** – responsible for defining project requirements, prioritizing features, and ensuring that the final software meets the functional and safety expectations.
- **ADAS feature developer (s)** – implement ADAS functionality and support verification, validation, and testing; independent from QA tasks related to day-to-day coding.
- **HMI/Interface Developer (s)** – responsible for dashboard design and interface quality.

In our organization we can subdivide the actors involved in the project, either external or internal :

- **Internal:** All team members collaborate through sprint planning, daily stand-ups, code reviews, and shared documentation (Git repository, issue tracker).
- **External:** Suppliers of sensors, STM32 modules, or other hardware components provide technical specifications and firmware updates (in our case, it is also the client). Communication is managed by all the team.

The software SPAP aligns with the overall system assurance approach. Software QA activities contribute directly to safety and reliability by ensuring correct ADAS behavior and sensor integration. Integration tests with hardware validate that software decisions translate correctly to actions (steering, speed, airbags).

Software product assurance activities are carried out collaboratively by the team members not directly involved in the feature being verified. For example, ADAS developers may review and test each other's modules, and HMI developers may validate interface-related functionality developed by others.

This approach ensures objectivity and a degree of separation between development and verification, even without a dedicated QA role. The Scrum Master oversees the process to ensure that verification and testing are properly planned and executed.

Any software components provided by external suppliers (e.g., STM32 firmware, sensor drivers) are accompanied by reports and documentation. When a component or resource provided for the project is faulty or not working as expected, we consult the professor (that is also our supplier when in need) for guidance, corrections, or replacement.

5.2. Reporting

For our team, reporting is integrated into the **agile ceremonies**:

- **Daily/weekly Stand-ups**: team members briefly report progress, issues, or risks related to software quality and integration.
- **Sprint Planning**: quality and verification tasks are included in the sprint backlog, and responsibilities are assigned.
- **Sprint Review / Demo**: the team reports completed features, verified modules, and any unresolved issues to the client and members of a jury in a presentation.

5.3. Quality models

The GEICar project applies agile-oriented quality models to ensure software meets functional, safety, and interface requirements.

- An user story is considered as Done when:
 - All the operational requirements are implemented
 - Acceptance criteria described in the quality control point
- We control the quality of our models with
 - Functional tests on a specific the feature as stated in the Validation Test section of the Project Plan using
 - Nominal Threshold and Data Value expected
 - Nominal Used Case
 - Consistency and repeatability of the result
 - SonarQube Code Analysis
 - Mandatory peer review before integrating a feature on the stable version of the system
 - Integration tests on the whole system with associated priority from the development plan [4.4]

5.4. Methods and tools

In the project we are using methods such as:

- **Agile/Scrum** is applied to plan the development through sprints with sprint review and regular team meeting.
- **Peer Review** is used to ensure minimal error in the stable version of the code, with team members reviewing each other's work before a feature is pushed in development.

- **Testing** is performed through acceptance tests to verify that modules and features work correctly before integration.

To facilitate the usage of these mentioned methods, we are using several tools such as:

- **Github** as our versioning tools to facilitate collaboration and tracking of code history. We set up a Pull Request workflow with a mandatory reviewer to merge in the develop branch. This ensures that the code is peer reviewed before being pushed in develop.
- **SonarQube Cloud** is used to analyze code quality in addition to peer review to ensure readability, maintainability and compliance to good coding practices
- **Jira** is used as a management software to track the tasks planned each sprint, backlogs and the responsibilities to each team member.
- **Google Drive** is used as a cloud storage to save our Project Plan, Development Plan, and all necessary documents for the project progress. In those documents, we have the Project Plan where all acceptance test are listed with numerical sensors value

6. Software process assurance

6.1. Projects plans

The SPAP references all project plans used throughout the software lifecycle:

- Project Plan: defines the overall schedule, milestones, stakeholders, and constraints.
- Software Development Plan (SDP): describes the engineering approach, Agile workflow, sprint organisation, version control strategies, tools, and roles.
- Software Product Assurance Plan (SPAP): defines the activities ensuring software quality, dependability, and compliance.

These plans are created progressively throughout the project:

- The Project Plan is produced during the sprint 0.
- The SDP and SPAP are started in Sprint 2 and a definitive version is given by Sprint 5.

All plans are updated at every sprint review, reflecting changes in the project according to the clients decisions and the features produced by the team.

Plans are maintained in the project's Google Drive Folder.

6.2. Software dependability and safety

Although the system is developed in a car model and not a real car, an extrapolation of criticality is applied:

ECSS-Q-ST-80 Analogous Category:

The ADAS driving functions (obstacle avoidance, trajectory control, emergency stop) could lead to hazardous vehicle behaviour if defective. The software would therefore fall under Category B, where a failure may cause limited harm or operational degradation.

The different cases where there is a failure and the consequences of that failure will be analyzed with a FMECA/AMDEC and a Fault Tree Analysis (FTA). Those documents are listed in our annexes.

DO-178C Equivalent Level:

Based on system impact and absence of life-critical operation, the functions correspond to Level D (Minor failure condition):

A failure could cause incorrect vehicle movement or loss of function. There would be no catastrophic outcomes as no one is driving the car and the car is too small to cause a major accident in the GEI.

6.3. Software documentation and configuration management

The project uses GitHub as the primary configuration management platform. To protect our software, only our team can modify our git repository. We also added some rules, so in our team only reviewers can push in the “develop” branch and main. We use a pull request system, so we can keep the history of the modification added in our project.

We use labels for our release in the main branch with the version number convention using `major.minor.fix`.

6.4. Process metrics

Code Quality Metrics (via SonarQube)

- Cyclomatic complexity
- Code duplication
- Comment density
- Technical debt
- Potential vulnerabilities and code smells

Agile Process Metrics

- Story point estimation using an action/difficulty matrix
- Burn-up charts updated at the end of each sprint
- Completion ratio of planned vs achieved tasks
- Team progress through bi-weekly progress reports

Metrics are collected using SonarQube CI analysis tools, and reviewed at each sprint review.

6.5. Reuse of software

The project reuses existing software components from a prior GeiCar repository. We also used a base repository for the HMI as a base. The reused components present no licensing constraints and were validated to match project needs.

6.6. Procedures and standards

The project uses GitHub for version control, collaboration and review platform. The project adopts a Pull-Request-based GitFlow workflow, tailored to ensure reliability and traceability with branch structure. Each branch is named `feature-<ID> - <feature_name>` and we have a hotfix branch to correct mistakes in the develop branch. We are following a classic GitFlow, with the develop branch as the main branch for our feature and a main branch for the release version of each sprints.

- Requirements captured through weekly client meetings.
- User stories and backlog managed on Jira with clear traceability.
- Validation at each sprint review through demonstrations.

We are coding the project in Python using ROS2 Middleware. We are applying good coding practice guidelines for the coding rules such as:

- explicit variable/function naming
- functions limited to ~20 lines when possible
- consistent indentation
- mandatory comments to describe the code
- line length limitation
- comment rate maintained per file/function
- Only simple floating-point operations (no direct equality comparison)

The metrics used are the default metrics set by SonarQube.

The Software is deployed directly on embedded platforms (Raspberry Pi / STM32) of the car using a python setup file. For the client validation, acceptance test is performed during all sprints review through scenario-based demonstrations.

7. Software product quality assurance

7.1. Quality Assurance Approach

The quality assurance of the Hold My Wheel software is based on an Agile workflow and a systematic verification of the different ADAS functionalities. The objective is to ensure that each feature is implemented safely, reliably, and in compliance with the system requirements. Quality is ensured through several mechanisms integrated into the development cycle:

- Definition of Done: a user story is completed only when code is reviewed, static analysis shows no critical issues, tests are successful, and the feature is demonstrated to tutors.
- Agile ceremonies: each sprint planning includes verification tasks and sprint reviews validate correct behaviour through demonstrations.
- Technical reviews with tutors: weekly sessions confirming architecture consistency, integration of sensors, and functional correctness of ADAS algorithms.

These activities ensure continuous monitoring of quality from design to final validation

7.2. Product metrics

The following metrics are used to verify that the software meets ADAS performance expectations for each one of the developed features. All target values come from system requirements and practical constraints.

- FCTA/RCTA: The correct detection of the distances between the vehicle and an obstacle is necessary to act on the motor's orders accordingly. (Speed to 30% of maximum speed when <1m, full stop between 20cm and 10cm)
- Airbag: The triggering function depends on the linear acceleration reading (the system must detect if it exceeds 0.4g on the X or Y axis)
- ESP: The trajectory deviation is detected by reading the angular velocity and the vehicle's cap (action must be taken when it exceeds 5 dps). Additionally, ESP Lite must be disabled when the vehicle is under 10% of its maximum speed.
- LDW: A good reading of distances between the car and the lane lines must be ensured (Alert if <5cm and stop doing so if >10cm)
- ACC: Distances between two cars and self linear speed must be correctly read to adjust the speed and remain between 80cm and 1m from the vehicle ahead.
- AES: Distance to an obstacle must be well read (Activation when obstacle < 50 cm).

Different approaches are used for data collection and analysis depending on the nature of the data.

- Automated: ROS2 logs (rosviz) are used for timestamps, distances, speeds and ADAS decisions. Moreover, SonarQube CI allows us to check the code quality. Finally, GitHub Actions are used for integration checks.
- Manual: physical measurement is used for the required distance values accuracy, and all scenario-based tests are previously validated with tutors.
- Storage: every test data is stored in Google Drive and reviewed at each sprint review.

Annexes

Safety and dependency analysis

AMDEC Analysis

Below is a table representing our AMDEC analysis on the different features of our car. The features have been decomposed into different functions and evaluated based on their risk, which encompasses a mixture of their severity level, their occurrence rate and their detectability. A high-risk item is considered when its risk level is above or equal to 5.

This AMDEC has been performed on a feature-by-feature basis. Each feature corresponds to the features highlighted in [Management objectives and priorities](#). For operational requirements see [Operational requirements](#).

Feature	Function		
[1] FCTA	Obstacle <1m	Failure	Obstacle not detected
		Effect	Collision
		Causes	Sensor failure, driver crash
		Risk (/10)	6
		Preventive measures	Startup self-test
		Corrective measures	Stop vehicle
	Measure distance	Failure	Distance wrongly estimated
		Effect	Stopping distance too short
		Causes	Noisy data, calibration error
		Risk (/10)	7
		Preventive measures	Filtering
		Corrective measures	Conservative breaking
	Safety logic	Failure	Incorrect safety decision

		Effect	Vehicle does not stop as required
		Causes	Logic error
		Risk (/10)	5
		Preventive measures	Unit tests, validation
		Corrective measures	Default to stop on error
[2] Airbag	Detect shock	Failure	Shock not detected
		Effect	Airbag not deployed during impact
		Causes	IMU failure, disconnection or driver saturation
		Risk (/10)	7
		Preventive measures	IMU self-test
		Corrective measures	Enter fault state
	Shock decision logic	Failure	Decision logic not triggered
		Effect	Airbag not deployed during impact
		Causes	Logic bug
		Risk (/10)	5
		Preventive measures	Unit tests, validation
		Corrective measures	
	False airbag deployment	Failure	Airbag deployed without shock
		Effect	Injury, system damage
		Causes	Noise, bug
		Risk (/10)	5
		Preventive measures	Multi-sample confirmation
		Corrective measures	Lockout after deployment

[3] ESP	Detect trajectory deviation	Failure	Deviation not detected
		Effect	Vehicle continues drifting
		Causes	Threshold misconfiguration, filtering delay
		Risk (/10)	5
		Preventive measures	Conservative thresholds
		Corrective measures	Alert user, reduce speed
	Trajectory deviation estimation	Failure	Deviation not properly estimated
		Effect	Incorrect correction
		Causes	Sensor noise, incorrect sampling
		Risk (/10)	7
		Preventive measures	Sensor fusion, validation
		Corrective measures	Limit speed
	Corrective decision logic	Failure	Incorrect corrective action
		Effect	Over or under correction, loss of control
		Causes	Control logic bug
		Risk (/10)	6
		Preventive measures	Simulation, unit tests
		Corrective measures	Disable correction, alert user
[4] LCA	Detect lanes	Failure	Lanes not properly identified
		Effect	Incorrect or absent correction
		Causes	Sensor failure, low visibility
		Risk (/10)	8
		Preventive measures	Sensor health monitoring

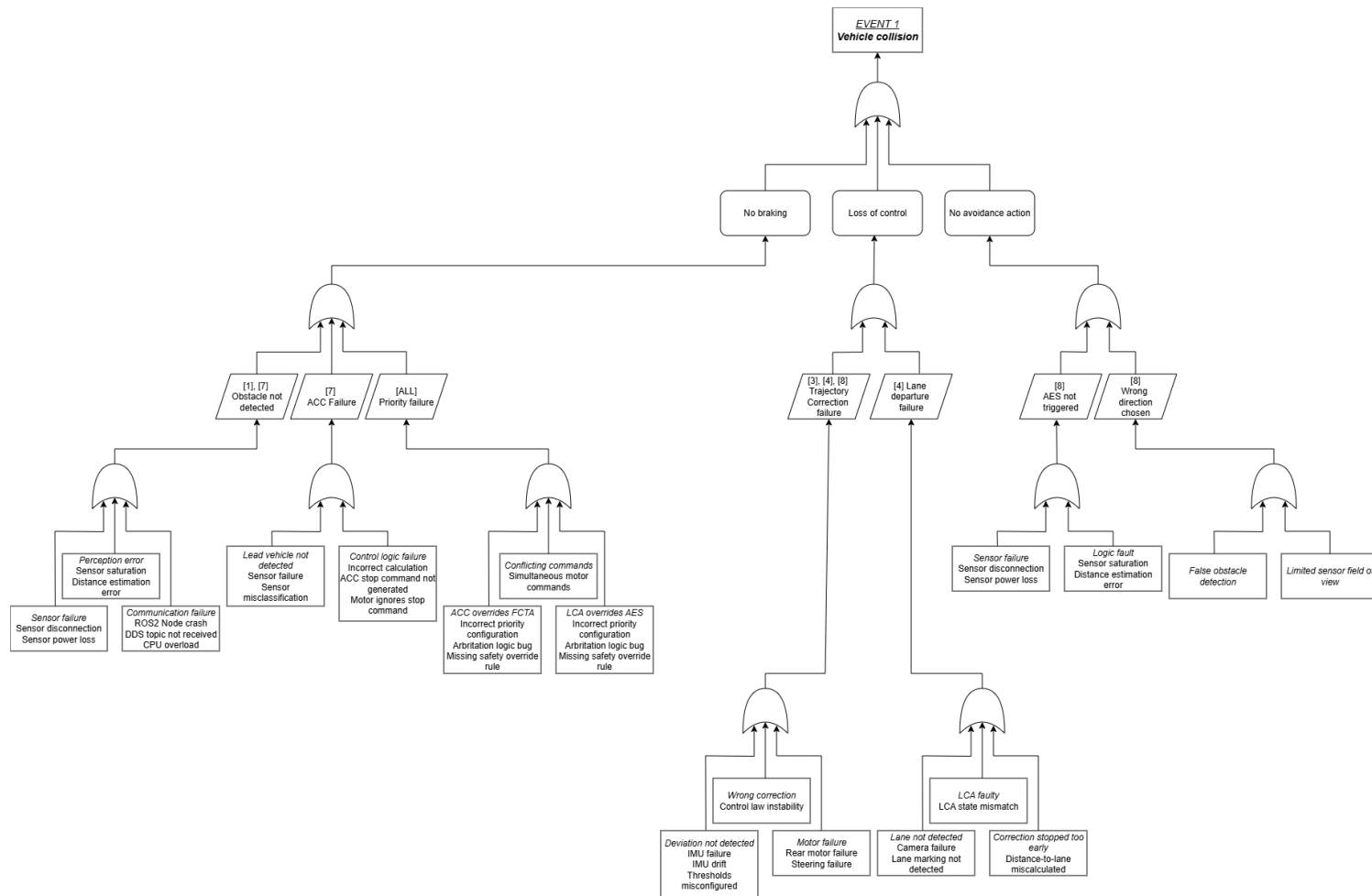
		Corrective measures	Disable LDW, alert user
	Estimate lane distance	Failure	Distance not properly estimated
		Effect	False lane departure
		Causes	Noisy data
		Risk (/10)	6
		Preventive measures	Filtering, plausibility checks
		Corrective measures	Increase warning margin
	Detect lane crossing	Failure	Lane crossing not detected
		Effect	No warning or correction
		Causes	Logic error
		Risk (/10)	6
		Preventive measures	State machine validation
		Corrective measures	Alert to uncertainty
	Trajectory correction	Failure	Incorrect correction
		Effect	Over or under compensation
		Causes	Logic bug
		Risk (/10)	5
		Preventive measures	Unit tests, validation
		Corrective measures	Disable correction, alert user
[5] HMI	Display system variables	Failure	Incorrect or missing display
		Effect	User misinformed
		Causes	UI crash, data loss
		Risk (/10)	7
		Preventive measures	Connection health monitoring

	Activate and deactivate ADAS via HMI	Corrective measures	Display fault indicator
		Failure	Wrong system activated
		Effect	Unexpected vehicle behavior
		Causes	UI bug
		Risk (/10)	4
		Preventive measures	Confirmation dialog
	Update HMI in real time	Failure	Frozen display
		Effect	Loss of awareness
		Causes	Communication failure
		Risk (/10)	5
		Corrective measures	Restart HMI Module
[6]	Detect failure	Failure	Failure not detected
		Effect	Feature compromised
		Causes	Sensor fault, disconnection or drift
		Risk (/10)	2
		Corrective measures	Disable related feature, alert user
[7] ACC	Maintain speed	Failure	Speed deviation
		Effect	Poor comfort, safety compromised
		Causes	Control error
		Risk (/10)	4
		Corrective measures	Limit acceleration
	Maintain a safe distance	Failure	Distance not respected
		Effect	Collision
		Causes	Motor failure, control error

		Risk (/10)	7
		Corrective measures	Emergency braking
	Priority handling	Failure	Wrong priority
		Effect	ACC overrides safety
		Causes	Priority management bug
		Risk (/10)	7
		Preventive measures	Unit tests, validation
		Corrective measures	Hard priority rules
[8] AES	Select free direction	Failure	Incorrect direction chosen
		Effect	Collision
		Causes	Sensor fusion error, logic bug
		Risk (/10)	5
		Preventive measures	Conservative choice
	Execute emergency steering	Failure	None or incorrect steering
		Effect	Loss of control
		Causes	Motor failure
		Risk (/10)	8
		Preventive measures	Reduce speed

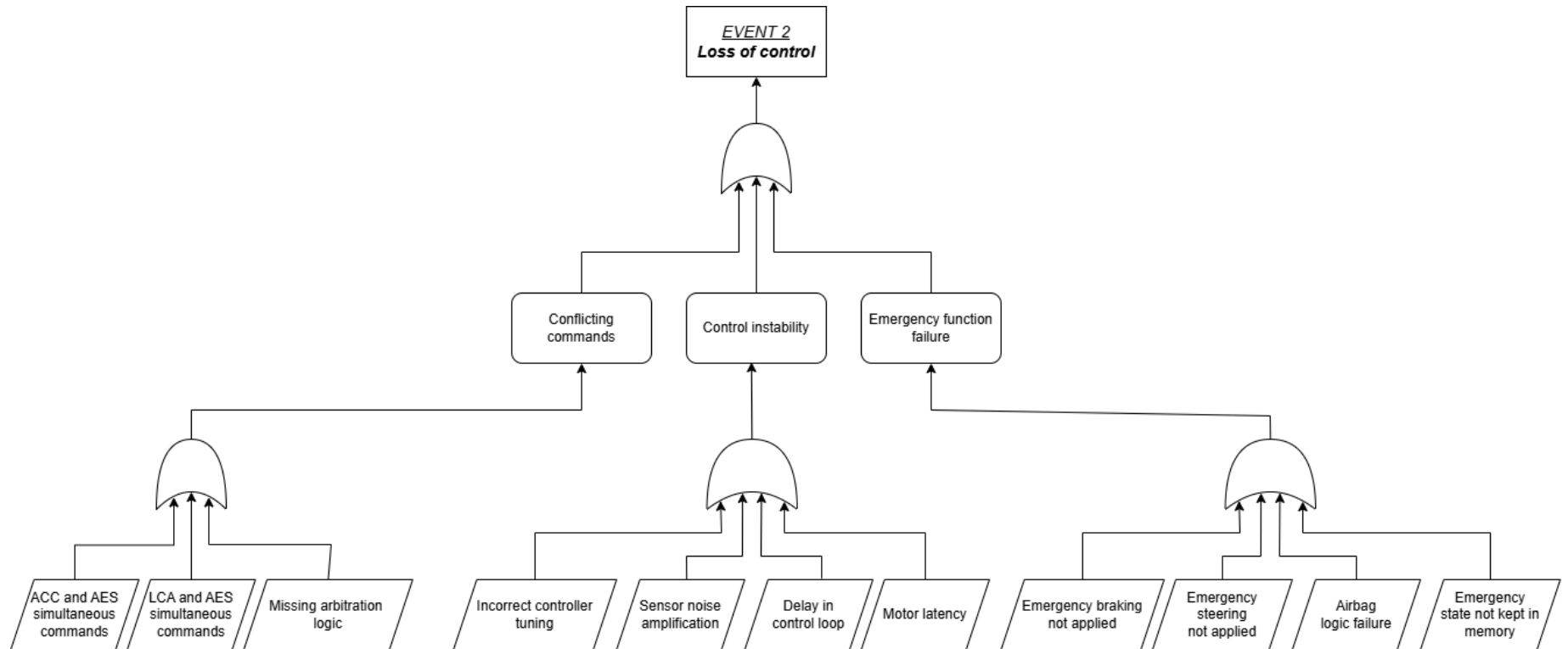
Fault tree

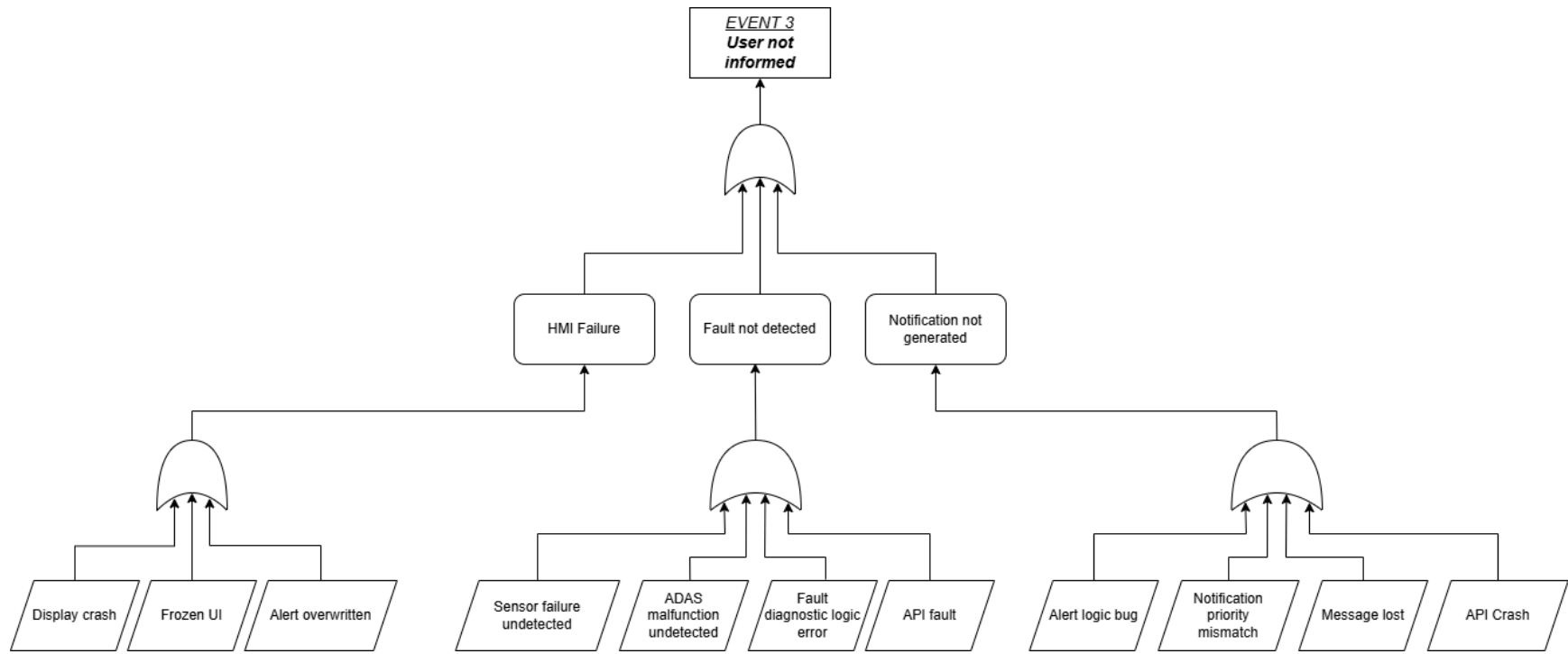
Contrary to the AMDEC analysis, our fault tree will focus on system-level outcomes compared to feature-level failures. The fault tree is divided into three big events : vehicle collision, loss of control and user not being informed.



5AE-SIEC - Hold My wheel

Events 2 and 3, being less critical than Event 1, are less detailed.





Operational requirements

Taken from the Project plan, the operational requirements outlined for our project are :

Feature ID	Req ID	Description	Level
[1]	[1.A.1]	The vehicle is capable of detecting any obstacle within 1 meter and displays the distance to that obstacle	Product
	[1.B.1]	If an obstacle is <20cm in front of the car, then the car must come to a complete stop, regardless of driver input	Sub-system
	[1.B.2]	Minimum stopping distance from obstacle >10cm	Sub-system
	[1.B.3]	If an obstacle is between 20 cm and 1 m in front of the car, the vehicle must limit its speed to no more than 30% of its maximum speed and alert the driver	Sub-system
[2]	[2.A.1]	The IMU is capable of detecting an acceleration/shock above 0.45g (similar to a strong curb hit) applied on both the front and sides.	Sub-system
	[2.B.1]	The vehicle needs to deploy the airbag when a shock has been detected	Sub-system
	[2.B.2]	The vehicle has to send a notification to the user on the HMI	Product
[3]	[3.A.1]	The vehicle must detect any deviation of its initial trajectory with a precision of ± 5 degrees per second.	Sub-system
	[3.A.2]	The system must alert the user of a loss of trajectory	Product
	[3.B.1]	The vehicle must try to get back to its initial trajectory	Sub-system
	[3.B.2]	The system must alert the user of a trajectory loss through a notification	Product
	[3.C.1]	The system must control the back motors and the direction independently for better trajectory control	Sub-system
[4]	[4.A.1]	The vehicle must detect when its distance is <5cm from a lane marking	Sub-system

	[4.B.1]	The user must be notified of lane crossing when LDW is enabled	Product
	[4.C.1]	The user must be able to activate and deactivate LDW using HMI	System
	[4.D.1]	The system must correct the trajectory until it is >10cm from the lane for more than 2 seconds	Sub-system
[5]	[5.A.1]	The HMI must show system variables such as speed, battery level	Product
	[5.A.2]	The HMI must show the status of the different systems	Product
	[5.B.1]	The system must be able to show user notifications and alerts coming from the other ADAS systems	Product
	[5.C.1]	The user must be able to deactivate or activate the different ADAS systems using the HMI	System
[6]	[6.A.1]	In the event of an ultrasonic sensor failure or of a FCTA/RCTA malfunction, the system must alert the user	System
	[6.B.1]	In the event of an accelerometer sensor failure or of an airbag malfunction, the system must alert the user	System
	[6.C.1]	In the event of a gyroscope sensor failure or of an ESP malfunction, the system must alert the user	System
	[6.D.1]	In the event of a camera failure or of a LDW/LKA malfunction, the system must alert the user	System
	[6.E.1]	In the event of a radar sensor failure or of an ACC malfunction, the system must alert the user	System
	[6.F.1]	In the event of a radar or ultrasonic sensor failure or of an AES malfunction, the system must alert the user	System
[7]	[7.A.1]	The system must stay within a range of $\pm 5\%$ of the desired speed	
	[7.B.1]	The user must be able to toggle the car's Automatic mode	System
	[7.B.2]	The user must be able to freely accelerate beyond the desired speed	Sub-system

	[7.B.3]	The desired speed must be set by the car's current speed when Automatic mode is activated	Sub-system
	[7.C.1]	The vehicle must detect any vehicle located <1m in front	Sub-system
	[7.D.1]	If no vehicle is detected at <1m, then the ACC must not alter the speed of the car	Sub-system
	[7.D.2]	If a car is detected <1m ahead of the car, then it must keep a safe distance between 80cm and 1m	Sub-system
	[7.D.3]	The user must be alerted if a car is detected <1m ahead of the car	Sub-system
	[7.D.4]	The car must fully stop if the vehicle ahead of it comes to a full stop	Sub-system
	[7.D.5]	ACC must have a lower priority than the FTCA	System
[8]	[8.A.1]	The AES must intervene on direction when an obstacle is <50cm away	Sub-system
	[8.A.2]	The AES must swerve in the direction where no obstacle is present.	Sub-system
	[8.A.3]	The AES must have priority over LDW and LKA	System
	[8.B.1]	The system must alert the user if it is about to intervene	Product